Git

03 - Workflow

📖 Support de cours

Si besoin, le support de cours est disponible sur la plateforme pédagogique.

Important : Vous trouverez également en annexe le document `Git-Synthèse.pdf` qui est indispensable à la réalisation de cette mission.

🚩 Objectifs de mission

Être capable de :

1. Comprendre les intérêts du *versioning*
2. Comprendre la notion de *repository* (*local* et *remote*)
3. Initialiser localement un projet Git
4. Associer un dépôt local à un dépôt distant
5. Définir un *workflow* simple
6. Comprendre et utiliser les commandes nécessaires à ce *workflow*

🏃 Mission

Question 1 - Compréhension

1. À quoi servent les commandes suivantes :
    - `git status`
    - `git branch`
    - `git checkout`
    - `git merge`
    - `git fetch`

Important : les réponses à cette question doivent être élaborées à partir de la documentation officielle :

- Git - git-status
- Git - git-branch
- Git - git-checkout

- Git - git-merge
- Git - git-fetch

2. Définir la notion de ***workflow***.
3. Préciser les avantages de définir un *workflow* dans le cadre du développement d'un projet.
4. Expliquer pourquoi il est habituel de distinguer une branche `main` d'une branche `develop`.
5. Expliquer le rôle du fichier `.gitignore`.
6. Expliquer l'intérêt de définir un fichier `README.md` et préciser le format auquel l'extension `.md` fait référence.

Pour ce travail, un **document à compléter** (`03_git_questions.docx`) est fourni en annexe.

Question 2 - Mise en place du *workflow*

Les manipulations à réaliser reposent sur les éléments présentés dans le **support de cours** ainsi que dans le **document de synthèse** fourni en annexe (`Git-Synthèse.pdf`).

A - Initialisation du projet

Effectuer dans l'ordre toutes les manipulations nécessaires à la mise en place sur votre poste de travail d'un projet pris en charge par Git et associé à un compte GitHub.

Pour cette partie, un modèle de fichier **`.gitignore`** est fourni en annexe.

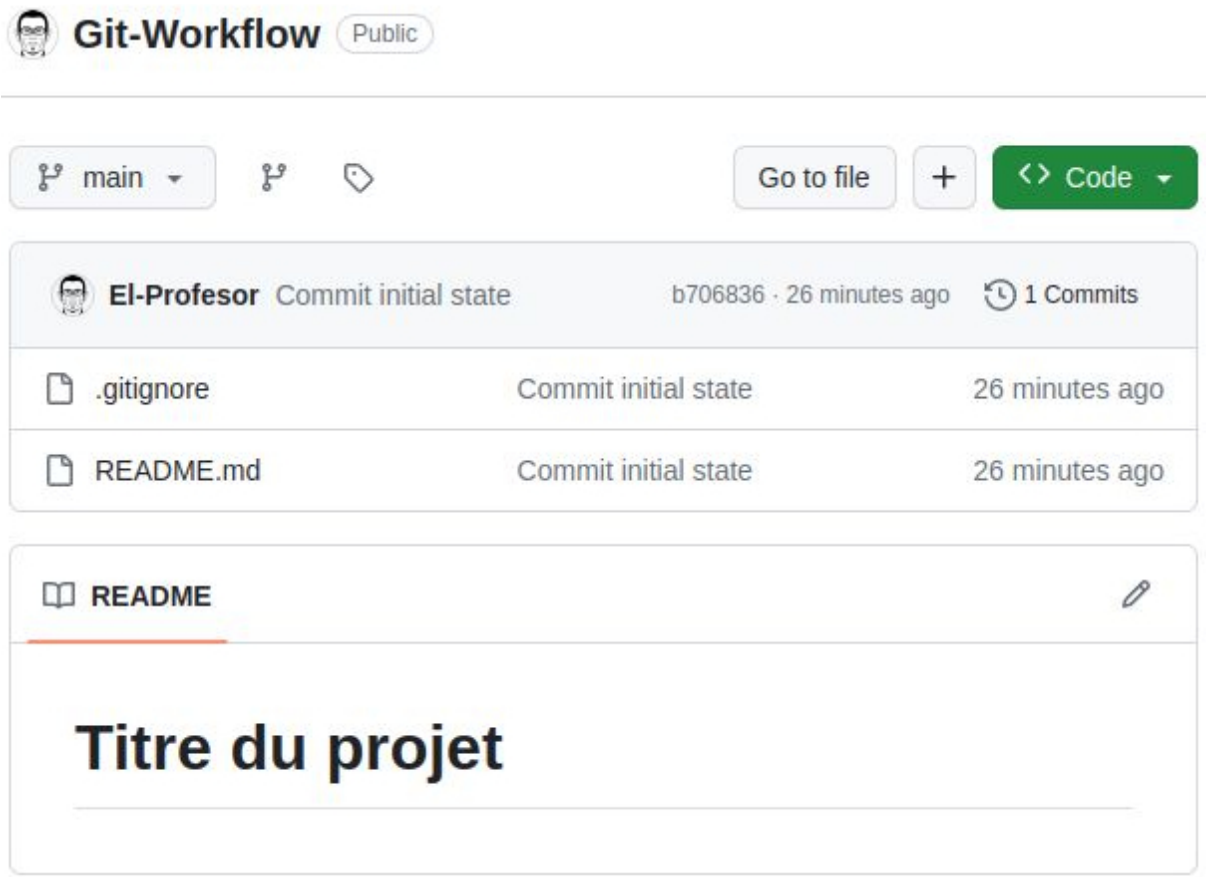
Le projet doit avoir comme nom : **Git-Workflow**.

Étapes à suivre (voir document de synthèse) :

1. Création du dépôt distant sur GitHub (voir mission précédente).
2. Création du dossier de projet.
3. Création du fichier `README.md` à la racine du projet.
4. Copie du fichier `.gitignore` à la racine du projet.
5. Initialisation de Git sur le dossier du projet (c.-à-d. mise en place du *versionning*).
6. Prise en compte des fichiers initiaux (`README.md` et `.gitignore`) dans le dépôt local.
7. Enregistrement via un *commit* avec ajout du commentaire associé.
8. Définition de la connexion au dépôt distant.
9. Mise à jour de la branche `main` distante.

Travail demandé :

- Faire au minimum trois captures d'écran :
  - la première montrant le formulaire de création du dépôt distant sur GitHub ;
  - la deuxième montrant l'ensemble des manipulations dans votre terminal ;
  - la dernière montrant le dépôt GitHub finalisé (voir exemple ci-dessous).



Remarques :

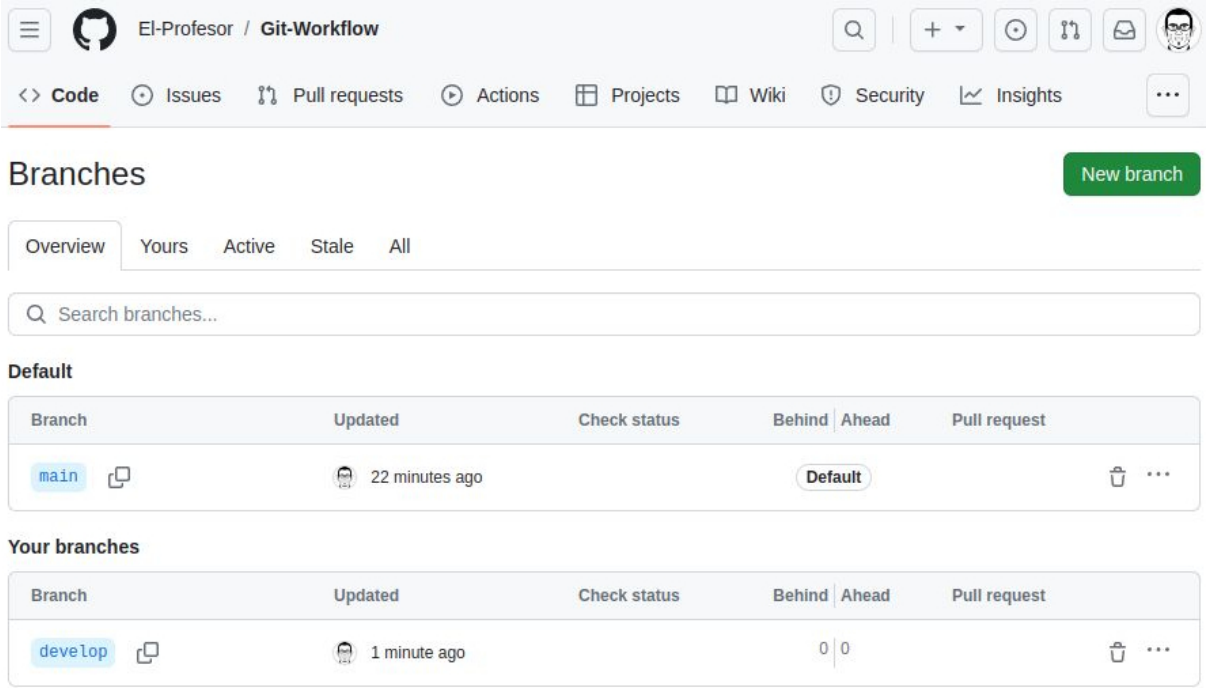
- Selon votre environnement de travail (Windows, Mac OS, ou Linux), vous pouvez être amené à adapter certaines commandes.

B - Préparation du projet pour la phase de développement

Effectuer les manipulations nécessaires afin de dériver la branche `develop` à partir de la branche `main`. Cette nouvelle branche doit être présente à la fois localement mais aussi sur le dépôt distant (GitHub).

Travail demandé :

- Faire une capture d'écran montrant la branche `develop` sur le dépôt GitHub (voir ci-dessous).



Question 3 - *Workflow*

Dans cette partie, il est question de pratiquer la *« routine »* de travail quotidienne basée sur le *workflow* présenté dans le support de cours.

Le travail doit être réalisé sous **Visual Studio Code** en ouvrant directement **le dossier** de travail `Git-Workflow/`.

A - Implémentation d'une fonctionnalité : initialisation

1. Avant toute chose, il est nécessaire de créer une branche locale de travail (ex. : `new_feature`) dérivée de `develop` afin de développer la nouvelle fonctionnalité (*feature*) :
    a. Ouvrir le terminal de VSCode
    b. Exécuter la commande adéquate

2. Une fois cette branche en place, créer un script `something.js` contenant le code suivant :

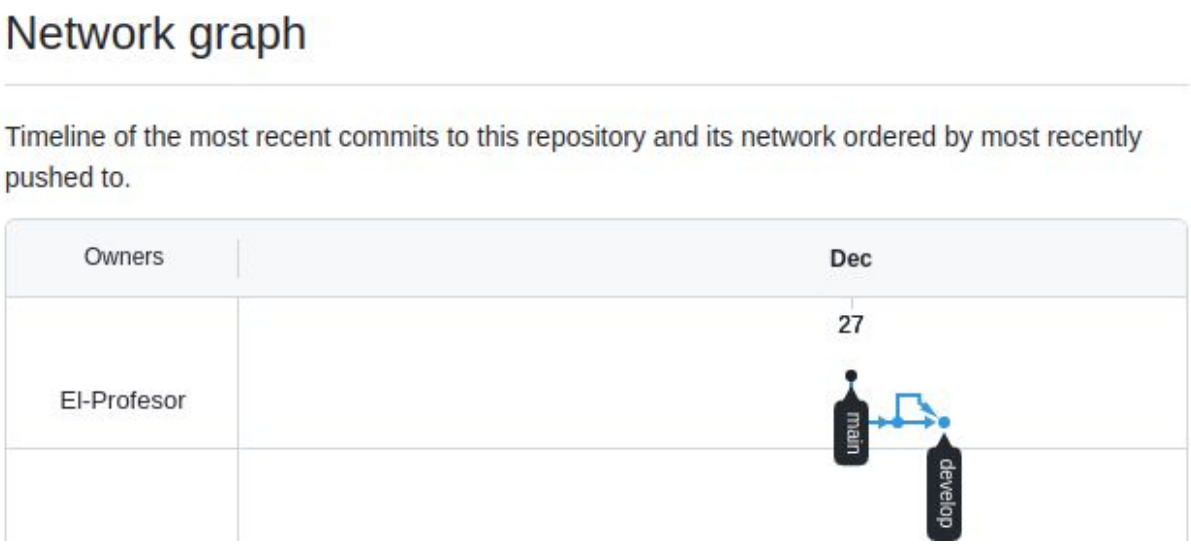
```
// Some useless JavaScript code

for (let i = 0; i < 10; i++) {
   console.log(i);
}
```

1. Vérifier l'état de l'arbre de travail (`git status`).
2. Ajouter les modifications apportées sur la branche de travail à la zone de transit (*staging area*).
3. Enregistrer ces modifications en spécifiant le commentaire associé au *commit* (ex. : *"Create new script"*)
4. Revenir sur la branche `develop`.
5. Faire le nécessaire pour mettre à jour le dépôt distant.
6. Une fois la mise à jour effectuée, penser à revenir sur la branche de travail.

Travail demandé :

- Faire une capture d'écran des commandes exécutées dans le terminal de VSCode.
- Faire une capture d'écran montrant l'arbre de versions sur le dépôt distant (GitHub > Page principale du dépôt > *Insights* > *Network graph*). Votre arbre de versions doit être identique à l'exemple suivant :



B - Implémentation d'une fonctionnalité : modification

Dans cette partie, il est demandé d'apporter des modifications successives au script `something.js`, d'**enregistrer à chaque étape** les modifications apportées sur la branche de travail, puis de mettre à jour, **après la dernière modification**, la branche `develop` distante.

La branche de travail reste la même que dans la partie précédente (ex. : `new_feature`).

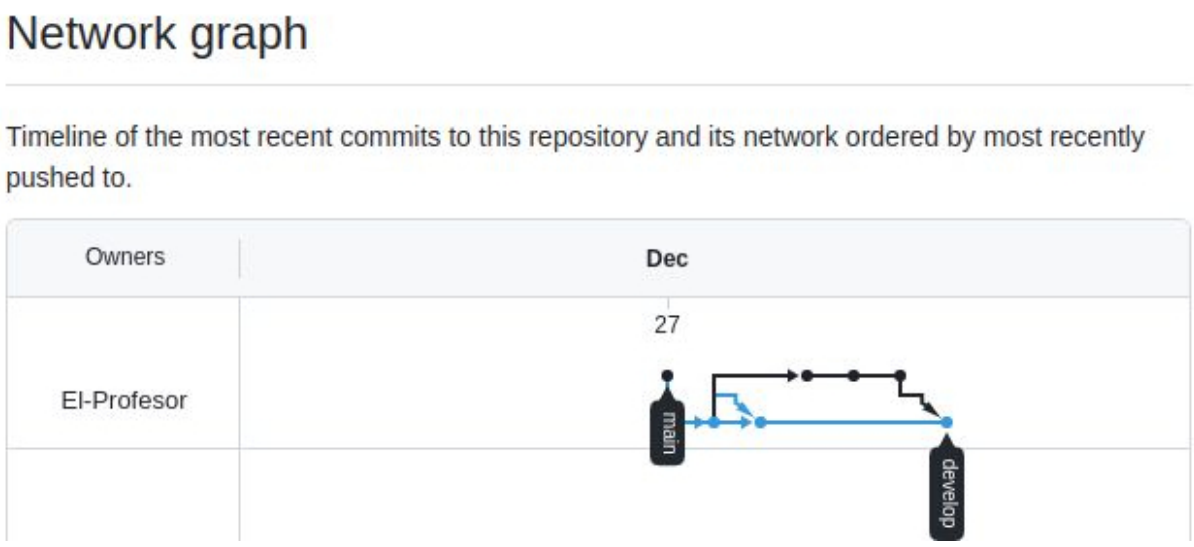
Voici la liste des modifications à effectuer sur le script `something.js` :

1. Remplacer le nom de variable `i` par `index`.

2. Créer deux variables `min` et `max` en amont de la boucle `for`, et affecter respectivement les valeurs `5` et `25` à ces variables.
3. Remplacer respectivement les valeurs *« en dur »* `0` et `10` de la boucle `for` par les variables `min` et `max`.

Travail demandé :

- Faire une capture d'écran des commandes exécutées dans le terminal de VSCode.
- Faire une capture d'écran montrant l'arbre de versions sur le dépôt distant. Ce dernier doit être identique à l'exemple suivant :



C - Implémentation d'une fonctionnalité : conclusion

Supposons que l'implémentation de la nouvelle fonctionnalité soit terminée. Il ne vous reste qu'à conclure proprement votre travail :

1. Si besoin, revenir sur la branche `develop`.
2. Supprimer la branche de travail utilisée jusque-là (ex. : `new_feature`).
3. Préparer une nouvelle branche de travail pour la fonctionnalité à venir (ex. : `next_feature`).

Travail demandé :

- Faire une capture d'écran des commandes exécutées dans le terminal de VSCode.
- Fournir le lien vers votre dépôt GitHub (`03_git_questions.docx`).
- Si vous avez bien saisi l'idée de *« routine »*, il suffit simplement de revenir au point de départ (voir section ***B - Implémentation d'une fonctionnalité : initialisation*** plus haut) : expliquer pourquoi.

Rendu de mission

Zip'n Drop ton dossier de travail contenant **le questionnaire** et **l'ensemble des *screenshots*** en le nommant :

`Prenom_Nom_03_Git.zip`